

src\case_studies\F_vtol\full_pd_controller.py

Generated: 2026-02-20 21:11:29

full_pd_controller.py

src\case_studies\F_vtol\full_pd_controller.py

```
# 3rd-party
import numpy as np

# Local (controlbook)
from .. import common
from . import params as P
from ..control.pd import PD
from ..control import utils_design

class VTOLControllerPD(common.ControllerBase):
    def __init__(self):
        # tuning parameters
        tr_h = 8.0 # original
        tr_h = 3.5 # tuned to just saturate for a single time step
        zeta_h = 0.707
        tr_theta = 0.8 # original
        tr_theta = 0.5 # tuned
        zeta_theta = 0.707
        M = 10.0 # time scale separation between inner and outer loop
        M = 7.0 # tuned
        tr_z = tr_theta * M
        zeta_z = 0.707

        # system parameters
        a1_lon, a0_lon = P.tf_lon_den[-2:]
        a1_lat_inner, a0_lat_inner = P.tf_lat_inner_den[-2:]
        a1_lat_outer, a0_lat_outer = P.tf_lat_outer_den[-2:]
        b0_lon = P.tf_lon_num[-1]
        b0_lat_inner = P.tf_lat_inner_num[-1]
        b0_lat_outer = P.tf_lat_outer_num[-1]

        # Longitudinal (altitude) gains
        wn_h = utils_design.get_natural_frequency(tr_h, zeta_h)
        alpha0_lon = wn_h**2
        alpha1_lon = 2 * zeta_h * wn_h
        kp_h = (alpha0_lon - a0_lon) / b0_lon
        kd_h = (alpha1_lon - a1_lon) / b0_lon
        self.h_pd = PD(kp_h, kd_h)
        print(f"longitudinal gains: {kp_h = :.3f}, {kd_h = :.3f}")

        ### Lateral (side-to-side) gains
        # inner loop (theta)
        wn_theta = utils_design.get_natural_frequency(tr_theta, zeta_theta)
        alpha0_lat_inner = wn_theta**2
        alpha1_lat_inner = 2 * zeta_theta * wn_theta
        kp_theta = (alpha0_lat_inner - a0_lat_inner) / b0_lat_inner
        kd_theta = (alpha1_lat_inner - a1_lat_inner) / b0_lat_inner
        self.theta_pd = PD(kp_theta, kd_theta)
        print(f"lateral inner loop gains: {kp_theta = :.3f}, {kd_theta = :.3f}")

        # DC gain of inner loop
        DC_gain = (b0_lat_inner * kp_theta) / (a0_lat_inner + b0_lat_inner * kp_theta)
        print(f"DC_gain = {DC_gain}") # a0 is 0, so DC_gain is 1

        # outer loop (z)
        effective_b0_outer = b0_lat_outer * DC_gain
        wn_z = utils_design.get_natural_frequency(tr_z, zeta_z)
        alpha0_lat_outer = wn_z**2
```

```

alpha1_lat_outer = 2 * zeta_z * wn_z
kp_z = (alpha0_lat_outer - a0_lat_outer) / effective_b0_outer
kd_z = (alpha1_lat_outer - a1_lat_outer) / effective_b0_outer
self.z_pd = PD(kp_z, kd_z)
print(f"lateral outer loop gains: {kp_z = :.3f}, {kd_z = :.3f}")

self.u_FT_eq = P.u_FT_eq
self.theta_max = np.radians(10)

def update_with_state(self, r, x):
    # unpack references and states
    z_ref, h_ref = r[:2]
    z, h, theta, zdot, hdot, thetadot = x

    # altitude (modified) PD
    F_tilde = self.h_pd.update_modified(h_ref, h, hdot)

    # Lateral outer Loop (modified) PD
    theta_ref = self.z_pd.update_modified(z_ref, z, zdot)
    # TODO: decide if we want to saturate the angle
    # theta_ref = self.saturate(theta_ref, self.theta_max)
    r[2] = theta_ref # if you want to visualize the "reference" angle

    # Lateral inner Loop (modified) PD
    tau_tilde = self.theta_pd.update_modified(theta_ref, theta, thetadot)

    # combine Lateral and Longitudinal control
    u_FT_tilde = np.array([F_tilde, tau_tilde])
    u_FT = u_FT_tilde + self.u_FT_eq

    # convert force-torque to motor forces
    u_unsat = P.mixer @ u_FT
    u = self.saturate(u_unsat, u_max=P.fr_max, u_min=0.0)
    return u

```